STUDENT TASK MANAGER

A DevOps Laboratory Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

MOHAMMED SOHAIL (24261A05A0)

PERUMANDLA TEJASHWIN (24261A05A8)

VINJAMUR KAUSHIK (24261A05C6)

Under the guidance of

K. SUNITHA

Faculty In-charge

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad-500075, Telangana, India (Autonomous)

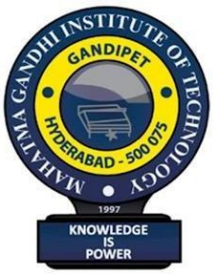
2026–2027

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad-500075, Telangana, India (Autonomous)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the DevOps laboratory project report entitled **"STUDENT TASK MANAGER"** is being submitted by **MOHAMMED SOHAIL** bearing Roll No. **24261A05A0**, **PERUMANDLA TEJASHWIN** bearing Roll No. **24261A05A8** and **VINJAMUR KAUSHIK** bearing Roll No. **24261A05C6** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, and is a record of bonafide work carried out by them in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, during the academic year 2026–2027, under our guidance and supervision.

Faculty In-charge
K. SUNITHA
Department of CSE, MGIT

Head of the Department
RAJNIKANTH
Department of CSE, MGIT

DECLARATION

We hereby declare that the work reported in this project report entitled **"STUDENT TASK MANAGER"** is a record of work carried out by us in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, Hyderabad, as part of the DevOps laboratory during the academic year 2026–2027. The application, configuration files, tests and evidence described in this report were produced by our team. Wherever material from documentation or other sources has been used, it has been duly acknowledged and referenced. This work has not been submitted elsewhere for the award of any degree or diploma.

MOHAMMED SOHAIL (24261A05A0)

PERUMANDLA TEJASHWIN (24261A05A8)

VINJAMUR KAUSHIK (24261A05C6)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without mentioning the people who made it possible. We take this opportunity to thank all those whose guidance and encouragement helped us complete this project.

We express our sincere thanks to the **Principal, Mahatma Gandhi Institute of Technology**, for providing the facilities and environment needed to carry out this work.

We are grateful to **Rajnikanth**, Head of the Department of Computer Science and Engineering, MGIT, for the constant support and encouragement extended to us throughout the DevOps laboratory.

We are deeply thankful to our faculty in-charge, **K. Sunitha**, Department of Computer Science and Engineering, MGIT, for the guidance, valuable suggestions and continuous support given to us, which helped us understand and apply DevOps practices in this project.

Finally, we thank all the faculty and staff of the Department of Computer Science and Engineering, and our families and friends, who helped us directly or indirectly in completing this project.

MOHAMMED SOHAIL (24261A05A0)

PERUMANDLA TEJASHWIN (24261A05A8)

VINJAMUR KAUSHIK (24261A05C6)

ABSTRACT

DevOps unifies software development and operations through collaboration, automation and continuous feedback. This project applies the core DevOps tool chain to **Student Task Manager**, a framework-free web application built with HTML, CSS and vanilla JavaScript that helps students organise their tasks. The application supports adding, completing and deleting categorised tasks, live task statistics, a persistent dark/light theme and a responsive layout. To create realistic user flows for automated testing, it was extended with a client-side authentication layer providing registration, login, logout, protected pages, profile editing, password change and separate task lists for each user. All data is stored in the browser's localStorage; there is no backend server or database, and the authentication is explicitly a demonstration-level mechanism rather than production-grade security.

Source code was managed with **Git and GitHub** using a feature-branch workflow, with three features integrated through pull requests and the second-phase work combined on an integration branch. A **Jenkins** declarative pipeline on a Windows agent checks out the repository, validates that the required files exist using batch commands, and archives the files with fingerprints; build #14 completed all stages successfully. The application was containerised with **Docker** using the nginx:alpine image, published on host port 8080, and orchestrated with **Docker Compose** together with a Selenium standalone Chrome service and a Python test-runner service, using profiles, service-name networking and a health check. An automated **Selenium WebDriver and Pytest** suite of 16 end-to-end test cases, designed from a shared contract before the implementation was written, covers signup validation, login, access control, profile management, task persistence, per-user task isolation and theme persistence. In the observed run, **all 16 tests passed in 34.01 seconds**.

The report documents the planning, implementation, CI pipeline, containerisation, testing, deployment and operation of the project with evidence from the repository, Jenkins, Docker and the test runs, and identifies its limitations and future enhancements such as backend authentication, a database, cloud deployment and running the browser tests within the CI pipeline.

Keywords: *DevOps, Git, GitHub, Jenkins, continuous integration, Docker, Docker Compose, Nginx, Selenium, Pytest, localStorage, test automation*

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Introduction to DevOps 1
1.2 Problem Statement and Motivation 1
1.3 Project Objectives 1
1.4 Scope of the Project 2
1.5 System Overview and Technologies Used 2
CHAPTER 2: PROJECT PLANNING AND VERSION CONTROL 4
2.1 Project Planning and Team Organisation 4
2.2 Git, GitHub and Branching Strategy 4
2.3 Commit History and Pull Requests 5
2.4 Merge Workflow in the Second Phase 6
CHAPTER 3: APPLICATION DEVELOPMENT 7
3.1 Application Architecture 7
3.2 Frontend and Task Management 8
3.3 localStorage Persistence 8
3.4 Authentication and Profile Management 9
3.5 Theme Switching 10
CHAPTER 4: CONTINUOUS INTEGRATION USING JENKINS 11
4.1 Continuous Integration and Jenkins 11
4.2 Jenkins Setup and Job Configuration 11
4.3 Pipeline Stages 11
4.4 Build Execution and Console Output 12
4.5 Extended Jenkinsfile on the Integration Branch 13
CHAPTER 5: CONTAINERISATION USING DOCKER 14
5.1 Containerisation and Docker 14
5.2 Dockerfile and Port Mapping 14
5.3 Docker Compose 14
5.4 Running Containers 15
CHAPTER 6: AUTOMATED TESTING USING SELENIUM AND PYTEST 16
6.1 Selenium WebDriver and Pytest 16
6.2 Test Environment and Execution Workflow 16
6.3 Test Case Design 16
6.4 Test Results and Analysis 17
CHAPTER 7: DEPLOYMENT, OPERATIONS AND MONITORING 19
7.1 Deployment Approaches 19
7.2 Local Deployment and Release Workflow 19
7.3 Monitoring and Troubleshooting 20
7.4 Problems Encountered and Solutions 20
CHAPTER 8: CONCLUSION AND FUTURE SCOPE 21
8.1 Conclusion 21
8.2 Limitations 21
8.3 Future Enhancements 21
REFERENCES 22

LIST OF FIGURES

Figure 1.1: Overview of the DevOps workflow implemented in the project 3
Figure 2.1: GitHub repository overview showing the main branch, commit count and github-pages deployments 5
Figure 2.2: GitHub branch overview showing the default and active branches 5
Figure 2.3: Branch and merge history reconstructed from the Git log 5
Figure 2.4: Merged pull request #3 (add tasks categories) on GitHub 6
Figure 3.1: Client-side architecture of Student Task Manager 7
Figure 3.2: Task dashboard after login, with personalised greeting and empty task list 8
Figure 3.3: Adding a new task with a category 8
Figure 3.4: Login page in dark theme (email address redacted) 10
Figure 3.5: Profile page with read-only email (redacted) and change-password form 10
Figure 3.6: Task dashboard in light theme with two pending tasks 10
Figure 3.7: Task dashboard in dark theme with the same tasks 10
Figure 4.1: Jenkins dashboard showing the Student-Task-Manager-CI job and its last success and failure 11
Figure 4.2: Pipeline stages executed in build #14 and their durations 12
Figure 4.3: Stage view of successful build #14 13
Figure 4.4: Console output of build #14 ending with Finished: SUCCESS 13
Figure 4.5: Extended Jenkinsfile on the integration branch (Validate stage excerpt) 13
Figure 5.1: docker-compose.yml with the web, selenium and tests services 15
Figure 5.2: Docker Compose services, network and published ports 15
Figure 5.3: docker ps showing the healthy Selenium container and the web container with 8080->80 15
Figure 6.1: Pytest session header and per-test results (16 items collected) 17
Figure 6.2: Short test summary showing 16 passed in 34.01 seconds 17

LIST OF TABLES

Table 1.1: Technologies used in the project 3
Table 2.1: Team members, responsibilities and supporting evidence 4
Table 2.2: Pull requests merged into main 6
Table 3.1: Application source files and their roles 7
Table 3.2: Validation rules and messages (from the contract, verified in auth.js) 9
Table 4.1: Summary of pipeline stages 12
Table 6.1: Test environment 16
Table 6.2: Selenium test cases and results 17
Table 7.1: GitHub Pages deployment compared with local Docker deployment 19
Table 7.2: Problems encountered and solutions implemented 20

CHAPTER 1: INTRODUCTION

1.1 Introduction to DevOps

DevOps is a set of practices, cultural values and tools that brings software development (Dev) and IT operations (Ops) together so that software can be built, tested, released and operated quickly and reliably. Instead of a slow, error-prone hand-over between separate teams, every change moves through the same automated and repeatable steps. The DevOps life cycle is a continuous loop of plan, code, build, test, release, deploy, operate and monitor, and each stage is supported by tools: Git and GitHub for version control and collaboration, Jenkins for continuous integration (CI), Docker for packaging an application together with its runtime, and Selenium for verifying behaviour from the user's point of view. The guiding principles are automation, collaboration through reviewed pull requests, frequent integration, reproducible environments described as code, and fast feedback.

1.2 Problem Statement and Motivation

Students juggle assignments, laboratory records, examinations and personal commitments, and benefit from a simple tool to track them. From the DevOps perspective the problem is broader: how can a small team develop such an application collaboratively, verify it automatically, package it so that it runs identically on any machine, and test it through a real browser using industry-standard tools? A deliberately small, framework-free application, Student Task Manager, was chosen so that the focus stays on the DevOps workflow. Because the original task features offered too few user flows for meaningful browser testing, a client-side authentication layer (signup, login, logout and profile management) was added, providing form validation, redirects and per-user data to test.

1.3 Project Objectives

1. To develop a responsive, framework-free Student Task Manager using HTML, CSS and JavaScript, with data persisted in browser localStorage.
2. To add client-side authentication, protected pages, profile editing, password change and per-user task lists.
3. To manage source code with Git and GitHub using feature branches and pull requests.

4. To configure a Jenkins declarative pipeline that checks out, validates and archives the project.
5. To containerise the application with Docker (Nginx) and orchestrate it and its test environment with Docker Compose.
6. To design and run an automated Selenium WebDriver and Pytest suite against the containerised application.

1.4 Scope of the Project

In scope: a static, client-side web application; localStorage for accounts, sessions, tasks and theme; Git and GitHub with branches and pull requests; a Jenkins pipeline on a Windows agent; Docker and Docker Compose for local deployment and testing; a 16-case Selenium suite; and static hosting through GitHub Pages. **Out of scope:** a backend or database server, production-grade authentication, cloud deployment of containers and advanced monitoring. The authentication is deliberately a demonstration-level mechanism: accounts live in the user's browser and passwords are transformed with a simple, non-cryptographic hash, so it must not be used for real accounts.

1.5 System Overview and Technologies Used

Student Task Manager consists of four HTML pages (signup.html, login.html, index.html and profile.html), one stylesheet (style.css) and two scripts (auth.js and script.js). After creating an account and logging in, a user can add tasks with a category (College, Personal, Work or Other), mark them completed, delete them and view live statistics of total, completed and pending tasks. The chosen light or dark theme is remembered, a profile page allows the name and password to be changed, and each user's tasks are stored separately. Figure 1.1 summarises the DevOps workflow built around the application, and Table 1.1 lists the technologies used.

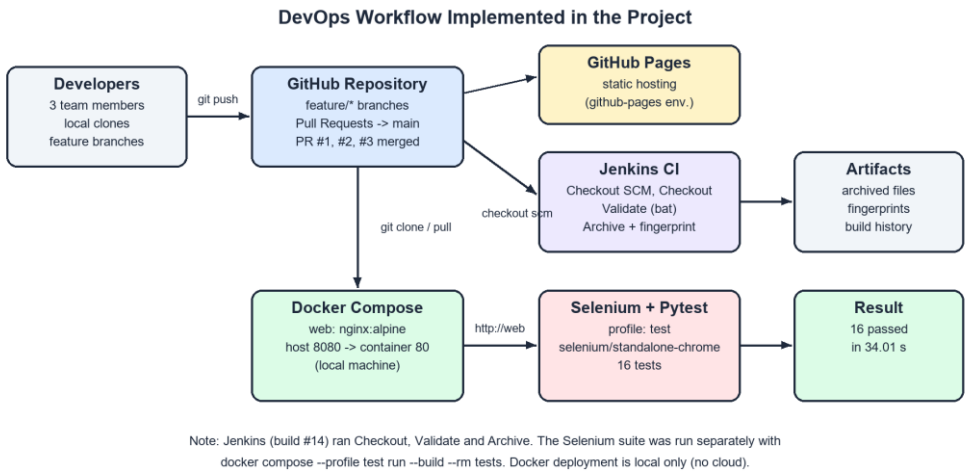


Figure 1.1: Overview of the DevOps workflow implemented in the project

Table 1.1: Technologies used in the project

Technology	Category	Purpose in this project
HTML5	Frontend	Structure of the four pages: signup, login, task dashboard and profile
CSS3	Frontend	Styling, CSS custom properties for light/dark themes, responsive layout
JavaScript (ES6, vanilla)	Frontend	Task logic, client-side authentication, theme switching, DOM rendering
Browser localStorage	Client storage	Persists users, current session, per-user tasks and theme on the user's device
Git	Version control	Tracks changes; feature branches and merges
GitHub	Collaboration / hosting	Remote repository, pull requests, GitHub Pages static hosting
Jenkins	Continuous integration	Declarative pipeline: Checkout, Validate, Archive on a Windows agent
Docker / nginx:alpine	Containerisation	Packages the static site into an Nginx web-server image
Docker Compose	Orchestration	Runs web, Selenium and test-runner services on one network
Selenium (standalone-chrome)	Test automation	Remote Chrome browser driven through WebDriver
Python 3.12, Pytest 8.4.2	Test framework	Test runner, fixtures and assertions (selenium>=4.20,<5.0)
Visual Studio Code	Development tool	Editing source files and configuration

CHAPTER 2: PROJECT PLANNING AND VERSION CONTROL

2.1 Project Planning and Team Organisation

The project was planned in two phases. In the first phase (September 2026) the team built the core task features, statistics, categories, a dark/light theme, GitHub Pages hosting and a Jenkins pipeline. In the second phase (late October to 1 November 2026) the team added authentication, Docker and Docker Compose, an automated Selenium suite, an extended Jenkinsfile and an updated README. For this phase a shared planning document, `00 shared context.md`, defined a contract that fixed the page names, the HTML element IDs used as Selenium locators, the exact validation messages, the localStorage keys and the Docker URLs. Because the contract was agreed in advance, the Selenium tests could be written in parallel with the authentication code. Table 2.1 shows the division of work and the repository evidence for it.

Table 2.1: Team members, responsibilities and supporting evidence

Member	Roll No.	Responsibilities	Repository evidence
Vinjamur Kaushik	24261A05C6	Project report and documentation; inspection and documentation of the DevOps workflow; evidence collection; initial project setup; Jenkins job configuration	Initial setup (e43f4a5); Jenkins pipeline fixes (853cd30, 55a6539, 559ce30); README updates; Jenkins build #14 "Started by Kaushik Vinjamur"; screenshots
Perumandla Tejashwin	24261A05A8	Task statistics and categories; authentication implementation (signup, login, logout); profile management and password change; client-side auth integration	c81f73a (statistics, PR #1); 1452f30 (categories, PR #3, merged by the same account); e03b8f1 (auth, branch feature/auth)
Mohammed Sohail	24261A05A0	Dark-mode feature; Dockerfile and Docker Compose; Selenium test implementation; Jenkins pipeline extensions; README updates; branch integration	Six dark-mode commits (PR #2); f17a511 and 9ee8c97 (feature/docker-selenium); merge ef9e7a6; Docker Hub tag sohails2409/student-task-manager

2.2 Git, GitHub and Branching Strategy

Git is a distributed version control system in which every developer holds the complete history; changes are recorded as commits, and branches are lightweight pointers that let several lines of development proceed independently. GitHub hosts the repository at https://github.com/vinjamurkaushik-web/student-task-manager and adds pull requests, review

and GitHub Pages hosting. Figure 2.1 shows the repository overview: the default branch main with 18 commits, six branches and a github-pages environment with eight deployments.

The team followed a feature-branch workflow with the rule "One branch per person. Never commit straight to main." Each feature was developed on its own branch (features/stats, feature/dark-mode, feature/auth and feature/docker-selenium) and integrated through a pull request. Figure 2.2 shows the branches on GitHub, and Figure 2.3 is a simplified drawing of the branch and merge history reconstructed from `git log --all --graph`.

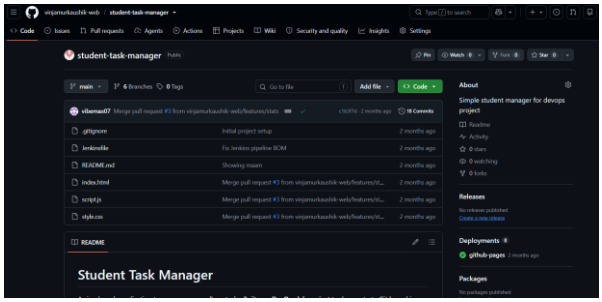


Figure 2.1: GitHub repository overview showing the main branch, commit count and github-pages deployments

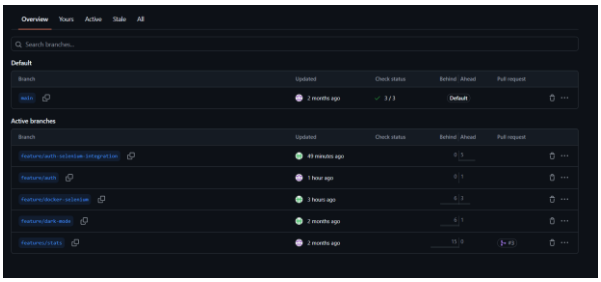


Figure 2.2: GitHub branch overview showing the default and active branches



Figure 2.3: Branch and merge history reconstructed from the Git log

2.3 Commit History and Pull Requests

The repository contains 23 commits across all branches. Author accounts map to team members as follows: Kaushik192006 and Kaushik2 – Vinjamur Kaushik; vibemax07 and "Perumandla Tejashwin" – Perumandla Tejashwin; RenNeedsSleep – Mohammed Sohail. The history shows good practice in places, such as the dark-mode work split into six small commits with conventional messages ("fix: apply saved theme before paint to prevent FOUC"), and weaker messages elsewhere ("New change now"). Three pull requests were merged into main (Table

2.2). Figure 2.4 shows pull request #3, which merged one commit changing three files with 34 added lines.

Table 2.2: Pull requests merged into main

PR	Source branch	Content	Merge commit	Merged on
#1	features/stats	Task statistics (total / completed / pending)	af91a41	6 Sep 2026
#2	feature/dark-mode	Dark/light theme toggle with persistence	b9f1bd6	10 Sep 2026
#3	features/stats	Task categories ("add tasks categories")	c1b3f7d	11 Sep 2026

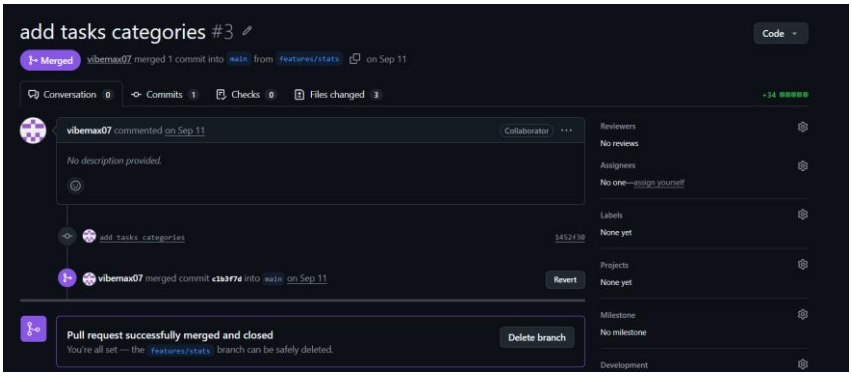


Figure 2.4: Merged pull request #3 (add tasks categories) on GitHub

2.4 Merge Workflow in the Second Phase

The shared context planned to merge feature/auth into main first and then rebase feature/docker-selenium. Because both branches were finished on the evening of 1 November 2026, the team instead created an integration branch, feature/auth-selenium-integration, and merged feature/auth into it (commit ef9e7a6, "Merge auth and Selenium branches"). The Selenium suite was run against this integrated code. At the time of capture the integration branch was five commits ahead of main with no pull request; the recommended next step is to open and merge one, so that main, GitHub Pages and the Jenkins job reflect the final application.

CHAPTER 3: APPLICATION DEVELOPMENT

3.1 Application Architecture

Student Task Manager is a purely client-side application with no application server and no database server. A web server (GitHub Pages or the Nginx container) only delivers static HTML, CSS and JavaScript; all behaviour runs in the browser, and all data is stored in localStorage, a key–value store scoped to the origin (scheme, host and port) on the user's own device. Each page loads auth.js, and the task dashboard additionally loads script.js (Figure 3.1). Table 3.1 summarises the source files.

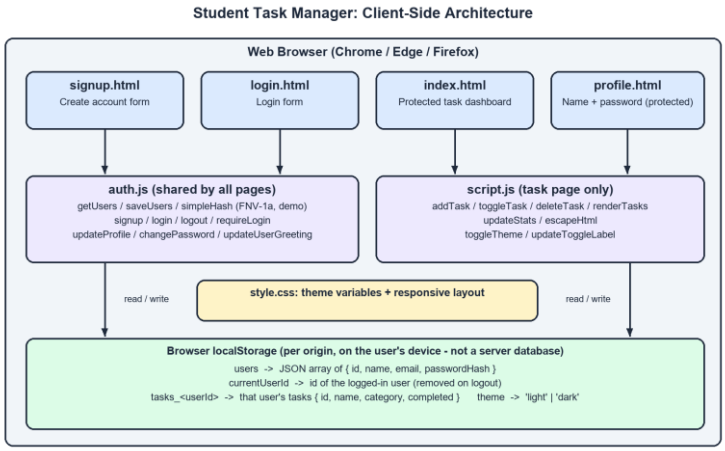


Figure 3.1: Client-side architecture of Student Task Manager

Table 3.1: Application source files and their roles

File	Lines	Role
index.html	91	Task dashboard. Applies saved theme before paint, loads auth.js and calls requireLogin(); contains task input, category selector, statistics cards and task list
login.html	41	Login form (login-email, login-password, login-submit, login-error) and link to signup
signup.html	50	Registration form (name, email, password, confirmation) with error and success messages
profile.html	64	Protected page with profile form (name, read-only email) and change-password form
auth.js	281	Shared authentication logic: user storage, demo hash, signup, login, logout, route protection, profile update, password change, greeting
script.js	219	Task operations, statistics, per-user persistence, HTML escaping, theme toggle
style.css	522	CSS custom properties for light and dark themes, card layout, forms, responsive rules

3.2 Frontend and Task Management

All pages use semantic HTML5 elements and a responsive viewport. Every interactive element has a stable `id` that forms part of the team contract, because the Selenium tests locate elements by ID; forms use `novalidate` so that the application's own messages, which the tests check word for word, are always shown. style.css defines all colours as CSS custom properties on `:root` and redefines them under `[data-theme="dark"]`, so switching the theme only toggles one attribute, and a media query adapts the layout for phones. The JavaScript is vanilla, with no frameworks or build tools.

A task is an object `{ id, name, category, completed }`. **addTask()** validates and adds a task, **toggleTask()** flips its completed state, **deleteTask()** removes it, **renderTasks()** rebuilds the list and the empty-state message, and **updateStats()** refreshes the three statistics cards. **escapeHtml()** encodes user input before it is inserted into the page, preventing script injection through task names. Figures 3.2 and 3.3 show the dashboard after login and a task being added.

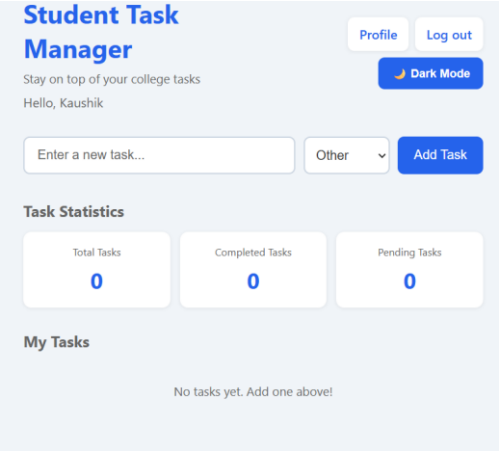


Figure 3.2: Task dashboard after login, with personalised greeting and empty task list

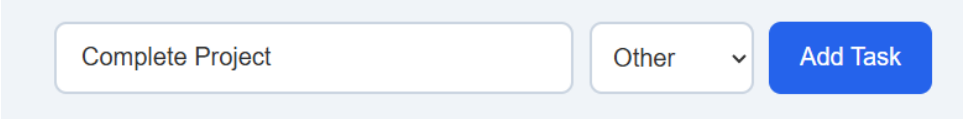


Figure 3.3: Adding a new task with a category

3.3 localStorage Persistence

localStorage stores strings only, so tasks and users are serialised with `JSON.stringify` and read back with `JSON.parse` inside `try...catch`, with checks that discard malformed entries so that corrupted storage cannot crash the page. Each user's tasks are kept under a separate key, `tasks_<userId>`, so two accounts in the same browser have completely separate task lists

(verified by test TC15). The other keys are `users` (all accounts with their password hashes), `currentUserId` (the active session, removed on logout) and `theme` (light or dark).

3.4 Authentication and Profile Management

Authentication was added on feature/auth (commit e03b8f1), which created auth.js, login.html, signup.html and profile.html. Passwords are never stored as entered: `simpleHash()` computes a 32-bit FNV-1a hash. The stronger Web Crypto API could not be used because it requires a secure context, and the Selenium browser reaches the application at the non-secure origin http://web. The hash is unsalted and offers no real protection, which is stated in the code, the README and Chapter 8.

`signup()` checks the input in a fixed order, shows the message of the first failing rule, stores the new user and redirects to the login page. `login()` shows the same "Invalid email or password" message for an unknown email or a wrong password, so the form does not reveal which accounts exist; on success it stores `currentUserId` and opens the dashboard. index.html and profile.html call `requireLogin()` in the page head and redirect to the login page when there is no session. The profile page updates the display name and changes the password after verifying the current one. Table 3.2 lists the validation messages, which match the contract exactly; Figures 3.4 and 3.5 show the login and profile pages.

Table 3.2: Validation rules and messages (from the contract, verified in auth.js)

Rule	Message displayed	Where
Any required field empty	All fields are required	Signup, login, profile, password
Email not of the form name@domain.tld	Enter a valid email address	Signup, login
Password shorter than 6 characters	Password must be at least 6 characters	Signup, change password
Confirmation does not match	Passwords do not match	Signup
Email already registered (case-insensitive)	Email already registered	Signup
Unknown email or wrong password	Invalid email or password	Login
Profile saved	Profile updated	Profile
Password changed	Password changed	Profile
Current password wrong	Current password is incorrect	Profile

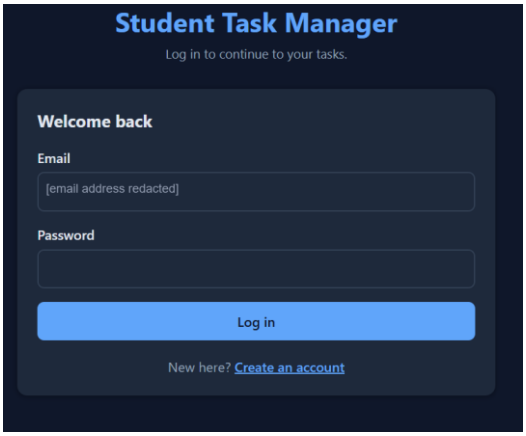


Figure 3.4: Login page in dark theme (email address redacted)

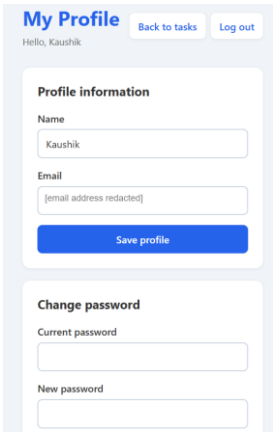


Figure 3.5: Profile page with read-only email (redacted) and change-password form

3.5 Theme Switching

The theme feature was developed on feature/dark-mode (PR #2). To avoid a "flash of unstyled content", a small inline script in each page's head applies the saved theme before the body is painted. `toggleTheme()` flips the `data-theme` attribute, saves the choice and updates the button label. Figures 3.6 and 3.7 show the same tasks in both themes.

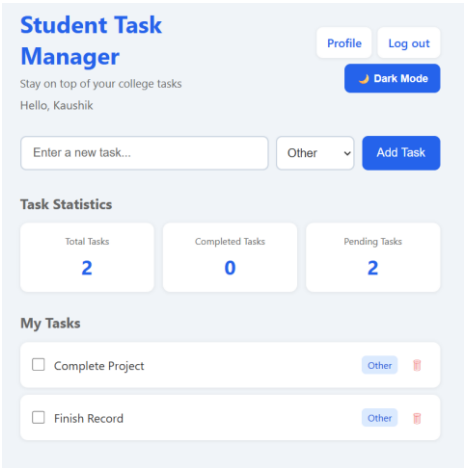


Figure 3.6: Task dashboard in light theme with two pending tasks

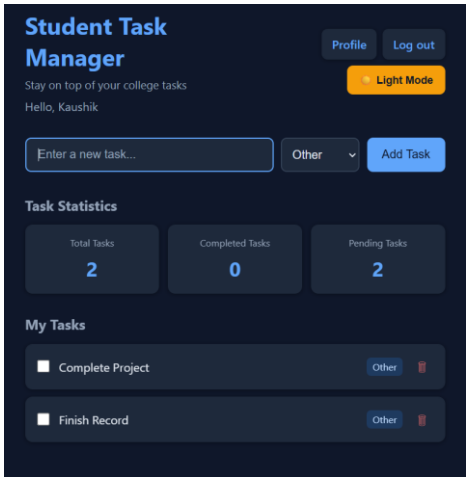


Figure 3.7: Task dashboard in dark theme with the same tasks

CHAPTER 4: CONTINUOUS INTEGRATION USING JENKINS

4.1 Continuous Integration and Jenkins

Continuous Integration (CI) is the practice of merging every developer's changes into a shared repository frequently and verifying each change automatically, so that problems are found minutes after they are introduced rather than weeks later. Jenkins is a widely used open-source automation server. A Jenkinsfile written in declarative pipeline syntax defines, as code stored in the repository, the agent, an ordered list of stages, the steps in each stage and optional post-build actions.

4.2 Jenkins Setup and Job Configuration

Jenkins was installed as a Windows service on a team member's computer (workspace `C:\ProgramData\Jenkins\.jenkins\workspace\Student-Task-Manager-CI`), with a Java runtime, the suggested plugins (including Pipeline and Git) and Git available to the service. A Pipeline job, Student-Task-Manager-CI, reads its definition from the GitHub repository ("Pipeline script from SCM"). Figure 4.1 shows the dashboard: the last success was build #13, the last failure build #3, and the last build took 7.7 seconds. The other jobs listed are not part of this project.

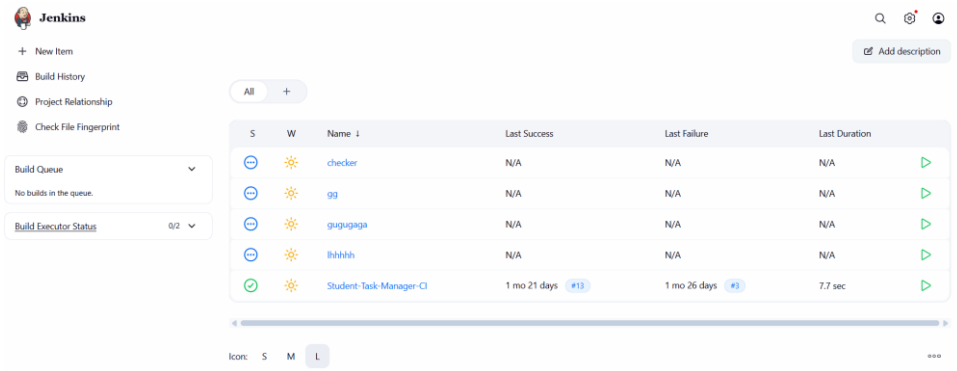


Figure 4.1: Jenkins dashboard showing the Student-Task-Manager-CI job and its last success and failure

4.3 Pipeline Stages

The Jenkinsfile on the main branch uses `agent any` and defines three stages. Jenkins adds an implicit Checkout SCM stage because the pipeline is loaded from source control, which makes

the explicit Checkout stage redundant but harmless. The Validate stage runs Windows batch commands such as `if not exist index.html exit /b 1`; any non-zero exit code fails the stage and the build. The Archive stage stores index.html, style.css, script.js and README.md with MD5 fingerprints, so the files of any build can be downloaded and traced later. Figure 4.2 and Table 4.1 summarise the stages.

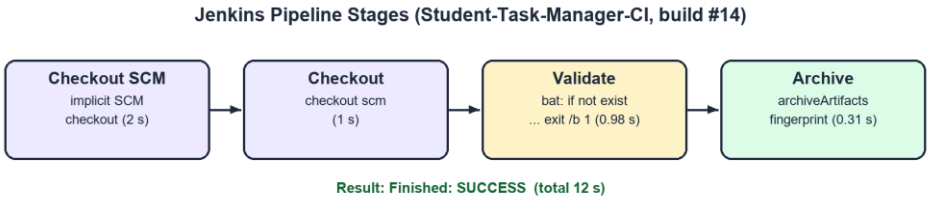


Figure 4.2: Pipeline stages executed in build #14 and their durations

Table 4.1: Summary of pipeline stages

Stage	Defined by	Action	Fails when	Build #14
Checkout SCM	Jenkins (implicit)	Clones the repository from GitHub	Repository unreachable	2 s, success
Checkout	Jenkinsfile	checkout scm	Repository unreachable	1 s, success
Validate	Jenkinsfile	bat: if not exist <file> exit /b 1	Any required file missing	0.98 s, success
Archive	Jenkinsfile	archiveArtifacts, fingerprint: true	No matching files	0.31 s, success

4.4 Build Execution and Console Output

Figure 4.3 shows build #14, started manually by Kaushik Vinjamur: all four stages succeeded in 12 seconds. The console output (Figure 4.4) shows the three file checks, the confirmation message, the archiving and fingerprinting of artifacts, and the final line Finished: SUCCESS. The earlier failure (build #3) was resolved by the commits "Fix Jenkins pipeline" and "Fix Jenkins pipeline BOM": a byte-order mark saved by a Windows editor prevented Jenkins from parsing the Jenkinsfile, and every later build succeeded.

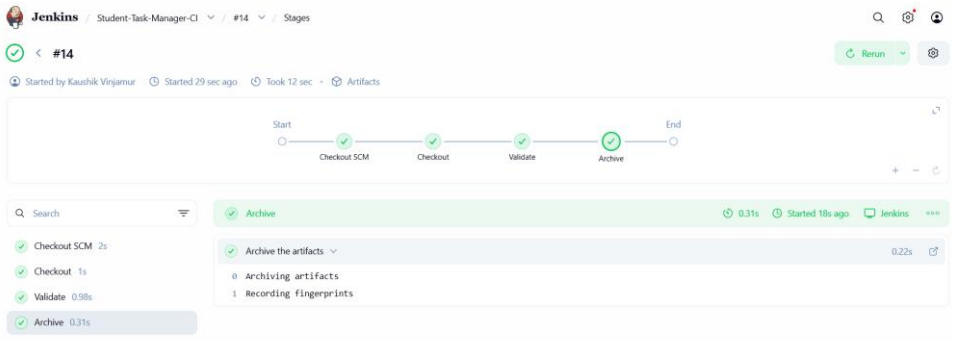


Figure 4.3: Stage view of successful build #14

4.5 Extended Jenkinsfile on the Integration Branch

On the integration branch the Jenkinsfile was extended (Figure 4.5). Validate now checks 17 application, Docker and test files; Archive includes the new files; a Docker Build stage runs `docker compose build web`; a Selenium Tests stage runs the Compose test profile with `--exit-code-from tests`; and a post action always runs `docker compose down`. The Jenkins job, however, builds main, and build #14 validated only the three main-branch files. No Jenkins run of the new stages appears in the evidence, and they also require Docker to be accessible to the Jenkins service. The Selenium suite was therefore run manually, and activating the extended pipeline is listed as future work.

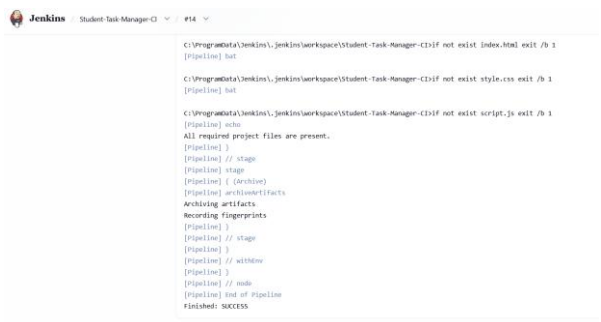


Figure 4.4: Console output of build #14 ending with Finished: SUCCESS



Figure 4.5: Extended Jenkinsfile on the integration branch (Validate stage excerpt)

CHAPTER 5: CONTAINERISATION USING DOCKER

5.1 Containerisation and Docker

Containerisation packages an application with its runtime, libraries and configuration into a portable container that shares the host kernel but runs in isolation and starts in seconds. Containers are created from images built in layers from a Dockerfile. Docker gave the team the same Nginx server on every machine, a reproducible test environment, and infrastructure as code versioned in Git.

5.2 Dockerfile and Port Mapping

Listing 5.1 shows the complete Dockerfile for the web application.

Listing 5.1: Dockerfile for the web application

```
# Use the small official Nginx image to serve this static website.
FROM nginx:alpine

# Copy every page and browser-side file into Nginx's public web directory.
COPY index.html login.html signup.html profile.html style.css script.js auth.js
/usr/share/nginx/html/

# Document that the container serves HTTP on port 80.
EXPOSE 80
```

`FROM nginx:alpine` uses the small official Nginx image, which serves static files without extra configuration. `COPY` places exactly the seven site files in Nginx's document root, and .dockerignore keeps .git, the tests and the documentation out of the build context. `EXPOSE 80` only documents the port; Docker Compose publishes it as `8080:80`, so the application opens at http://localhost:8080 without administrator rights, and `docker compose down` stops and removes the containers and network.

5.3 Docker Compose

docker-compose.yml (Figure 5.1) defines three services. **web** builds the Dockerfile, is tagged sohails2409/student-task-manager:latest and publishes 8080:80. **selenium** runs selenium/standalone-chrome with 2 GB of shared memory and publishes 4444 for WebDriver and 7900 for a noVNC live view. **tests** builds the Python 3.12 test runner from ./tests. The selenium and tests services belong to the `test` profile, so `docker compose up --build` starts only the web application, while the test command starts all three.

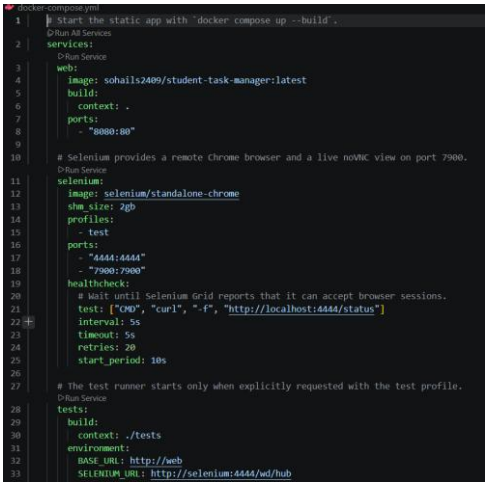


Figure 5.1: docker-compose.yml with the web, selenium and tests services

Compose creates a private network on which services reach each other by name (Figure 5.2): the test runner connects to http://selenium:4444/wd/hub, and the browser loads the application from http://web. Because a started container is not necessarily ready, the selenium service has a health check (`curl -f http://localhost:4444/status` every 5 seconds, up to 20 retries), and the tests service waits for `service_healthy` before running.

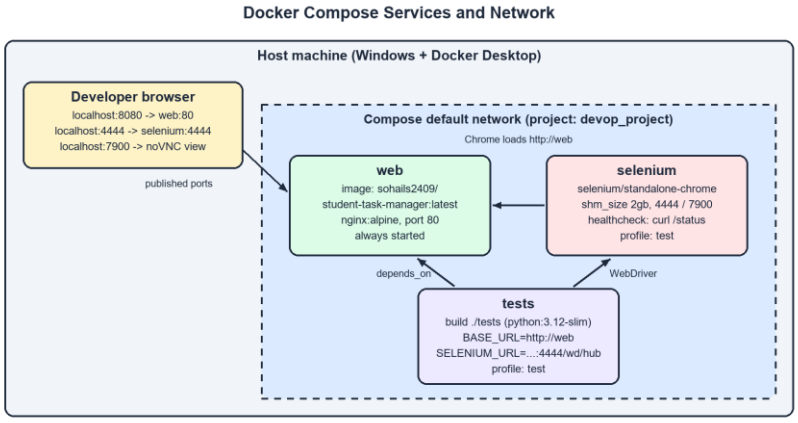


Figure 5.2: Docker Compose services, network and published ports

5.4 Running Containers

Figure 5.3 shows `docker ps` during the test session: devop_project-selenium-1 is up and healthy with ports 4444 and 7900 published, and devop_project-web-1 maps host port 8080 to container port 80. The test runner does not appear because it is removed after each run (`--rm`).

```
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS                    PORTS
                                                                   NAMES
c72adea93c83   selenium/standalone-chrome   "/opt/bin/entry_poin…"   3 minutes ago    Up 3 minutes (healthy)   0.0.0.0:4444->4444/tc
p, [::]:4444->4444/tcp, 0.0.0.0:7900->7900/tcp, [::]:7900->7900/tcp   devop_project-selenium-1
a36c2f5d4f53   c9e43a1d0b2c               "/docker-entrypoint.…"   17 minutes ago   Up 12 minutes             0.0.0.0:8080->80/tcp,
 [::]:8080->80/tcp                                                 devop_project-web-1
PS C:\Devop_Project>
```

Figure 5.3: docker ps showing the healthy Selenium container and the web container with 8080->80

CHAPTER 6: AUTOMATED TESTING USING SELENIUM AND PYTEST

6.1 Selenium WebDriver and Pytest

The project uses automated end-to-end functional tests that drive a real Chrome browser through the application, fill in forms, click buttons and check messages, URLs and stored state. Selenium WebDriver is the W3C-standard API for sending such commands to a browser; here the browser runs in the selenium container and the tests connect to it with `webdriver.Remote`. Explicit waits (`WebDriverWait`, 5-second timeout) are used instead of fixed sleeps, which makes the tests faster and more reliable. Pytest discovers and runs the tests, and a `browser` fixture in tests/conftest.py opens a fresh remote Chrome session for every test and always closes it, so each test starts with empty localStorage and cannot affect the others.

6.2 Test Environment and Execution Workflow

Table 6.1 summarises the environment of the observed run. The command `docker compose --profile test run --build --rm tests` builds the test-runner image, starts the web and selenium services, waits for the Selenium health check, runs the 16 tests and removes the container; it exits with status 0 only if every test passes.

Table 6.1: Test environment

Component	Value
Application under test	web service (nginx:alpine) at http://web inside the Compose network
Browser	Google Chrome in selenium/standalone-chrome, WebDriver at http://selenium:4444/wd/hub
Test runner	python:3.12-slim container; Python 3.12.16, pytest 8.4.2, pluggy 1.7.0 (platform linux)
Libraries	selenium>=4.20,<5.0; pytest>=8.0,<9.0
Command	docker compose --profile test run --build --rm tests
Code version	feature/auth-selenium-integration (auth merged with Docker/Selenium work)

6.3 Test Case Design

The tests were designed from the team contract before the authentication code was written, so the contract, not the implementation, defined the expected behaviour, and exact message strings

are asserted. They form four groups: signup (TC01–TC06, one positive case and one per validation rule), login and access control (TC07–TC11), profile (TC12–TC13), and tasks and theme (TC14–TC16), including the isolation of tasks between two users.

6.4 Test Results and Analysis

Figure 6.1 shows the Pytest session: 16 items were collected and each was reported as PASSED. Figure 6.2 shows the final summary, 16 passed in 34.01 s, about 2.1 seconds per test including a new Chrome session each time. Table 6.2 records each case with its expected and actual result.

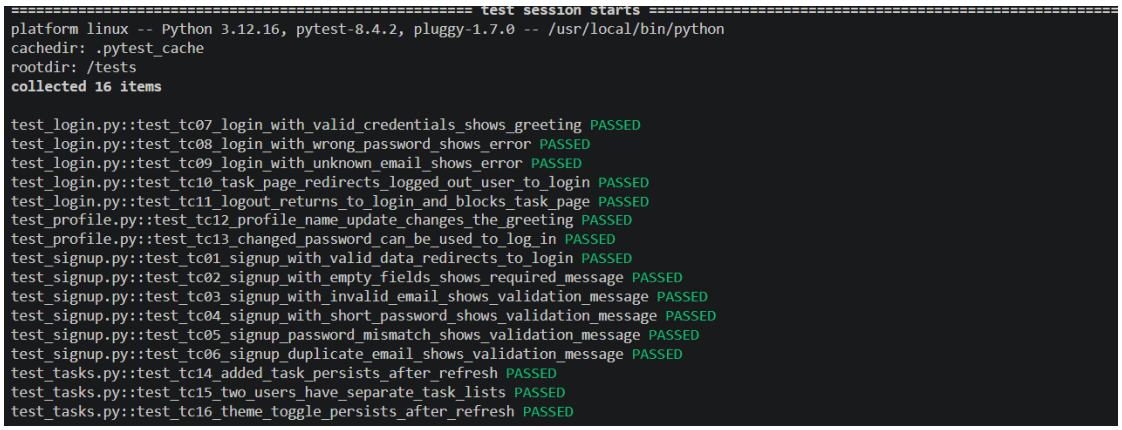


Figure 6.1: Pytest session header and per-test results (16 items collected)

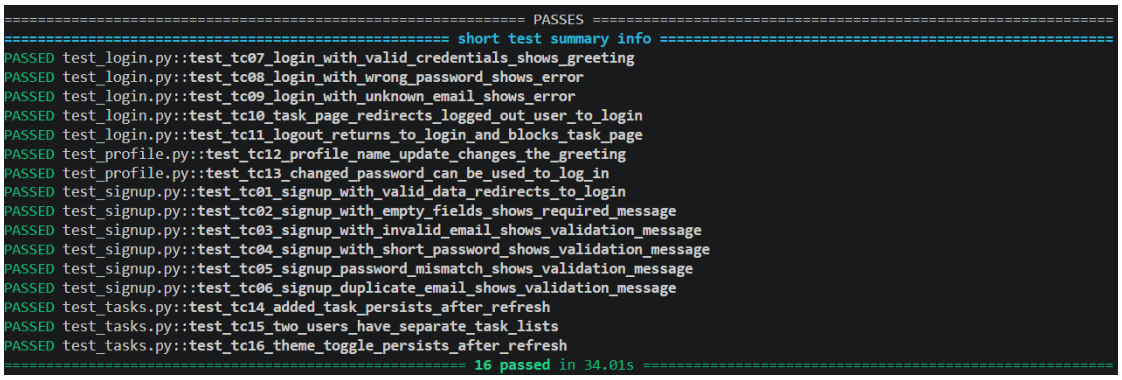


Figure 6.2: Short test summary showing 16 passed in 34.01 seconds

Table 6.2: Selenium test cases and results

ID	Test description	Expected result	Actual result	Status
TC01	Sign up with valid name, email and password	Redirect to login.html	As expected	PASS
TC02	Submit the signup form with all fields empty	"All fields are required"	As expected	PASS
TC03	Sign up with email "not-an-email"	"Enter a valid email address"	As expected	PASS

ID	Test description	Expected result	Actual result	Status
TC04	Sign up with 5-character password "short"	"Password must be at least 6 characters"	As expected	PASS
TC05	Sign up with a different confirmation password	"Passwords do not match"	As expected	PASS
TC06	Register the same email twice	"Email already registered"	As expected	PASS
TC07	Log in with valid credentials	index.html opens; greeting is not empty	As expected	PASS
TC08	Log in with a wrong password	"Invalid email or password"	As expected	PASS
TC09	Log in with an unregistered email	"Invalid email or password"	As expected	PASS
TC10	Open index.html without logging in	Redirect to login.html	As expected	PASS
TC11	Log out, then open index.html again	login.html shown; task page blocked	As expected	PASS
TC12	Change the profile name and return to tasks	"Profile updated"; greeting shows new name	As expected	PASS
TC13	Change password, log out, log in with new password	"Password changed"; login succeeds	As expected	PASS
TC14	Add a task and refresh the page	Task is still listed	As expected	PASS
TC15	Two users each add a task; switch accounts	Each user sees only their own task	As expected	PASS
TC16	Toggle to dark theme and refresh	data-theme remains "dark"	As expected	PASS

The results show that every validation message in the contract is produced exactly, redirects and access control work, profile and password changes take effect, and tasks and the theme persist across reloads with tasks isolated per user. Because both branches were built against the same contract, their integration worked as intended. The results come from a single manual run on one machine with Chrome and cover functional behaviour only, not performance, accessibility, security or other browsers.

CHAPTER 7: DEPLOYMENT, OPERATIONS AND MONITORING

7.1 Deployment Approaches

Two deployment approaches are used (Table 7.1). GitHub Pages provides public static hosting of the repository files and redeploys automatically when the configured branch changes; the repository shows eight deployments, the latest in September 2026, so the published site is expected to be the first-phase application without the login pages. Docker provides a local, containerised deployment for demonstration and testing; no container was deployed to a cloud platform.

Table 7.1: GitHub Pages deployment compared with local Docker deployment

Aspect	GitHub Pages	Local Docker deployment
Where it runs	GitHub's static hosting infrastructure	Team member's Windows computer (Docker Desktop)
Who can access it	Anyone with the public URL	The local machine at http://localhost:8080
Web server	GitHub Pages	Nginx (nginx:alpine) inside the web container
Source of files	Repository branch configured in GitHub Pages settings	Files copied into the image by the Dockerfile from the local checkout
Evidence	github-pages environment with 8 deployments (Figure 2.1)	docker ps (Figure 5.3); application screenshots (Chapter 3)
Includes authentication?	Only after the integration branch is merged into the deployed branch	Yes – built from feature/auth-selenium-integration

7.2 Local Deployment and Release Workflow

With Docker Desktop running, `docker compose up --build` builds the web image, creates the network and starts Nginx, which serves the static files on port 80 inside the container. The 8080:80 mapping (confirmed in Figure 5.3) makes the application available at http://localhost:8080, where a visitor who is not logged in is redirected to the login page. Because localStorage is scoped per origin, data created there is separate from GitHub Pages.

The release workflow is: develop on a feature branch; integrate and verify with Docker Compose and the Selenium suite; merge to main through a pull request; let Jenkins validate and archive the build; and let GitHub Pages republish. The image can optionally be pushed

with `docker compose push web`, although the evidence does not show this. The repository has no tags yet, so tagging v1.0 after the final merge is recommended.

7.3 Monitoring and Troubleshooting

Container state is inspected with `docker ps`, `docker compose ps` and Docker Desktop. The only automated health check is the Selenium Grid check; the availability of the web container is confirmed in the browser and implicitly by every passing test. Jenkins provides build monitoring: the dashboard shows status and stability icons, the times of the last success and failure and the build duration, and the console output gives a full log. Other log sources were `docker compose logs`, the verbose Pytest output, the noVNC live view at http://localhost:7900 and the browser developer tools. There is no metrics collection, centralised logging, alerting, Jenkins notification or uptime monitoring; these are listed as future enhancements.

7.4 Problems Encountered and Solutions

Table 7.2 lists the main problems identified from the documentation, the commit history and the source code, and how they were solved.

Table 7.2: Problems encountered and solutions implemented

Problem	Cause	Solution implemented
Jenkins pipeline failed (last failure: build #3)	Jenkinsfile problems, including a byte-order mark (BOM) at the start of the file	Commits "Fix Jenkins pipeline" and "Fix Jenkins pipeline BOM"; later builds succeeded
Theme briefly showed light before switching to dark (FOUC)	Theme was applied by script after the page was painted	Inline script in <head> applies data-theme before paint (commit c058ddf)
crypto.subtle unavailable in Selenium tests	Web Crypto requires a secure context; http://web is not secure	Hand-written demonstration hash (FNV-1a), documented as not secure
Tests could start before the browser was ready	Selenium Grid needs time to initialise	Compose health check on /status and depends_on: service_healthy
Timing-dependent test failures	Signup redirects after a 1 s delay; pages load asynchronously	WebDriverWait explicit waits instead of sleep
Tests affecting each other through stored data	localStorage persists within a browser session	New remote Chrome session per test in the browser fixture
Chrome crashes in containers with small shared memory	Docker's default /dev/shm is 64 MB	shm_size: 2gb on the selenium service

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The project achieved its objectives. The team built a responsive Student Task Manager with categorised tasks, live statistics, persistent themes, client-side authentication, profile management and per-user task lists, and implemented a DevOps workflow around it: Git and GitHub with feature branches and three merged pull requests; a Jenkins declarative pipeline that validates and archives the project; an Nginx-based Docker image; a Docker Compose environment with profiles, service-name networking and a health check; and a contract-driven Selenium and Pytest suite of 16 end-to-end tests, all of which passed in 34.01 seconds. The project shows that even a small application benefits from a disciplined DevOps workflow: feature branches and a written contract let members work in parallel, Docker made the environment reproducible, and automated browser tests gave objective evidence that the integrated application behaves as specified.

8.2 Limitations

- **Security and storage:** authentication is client-side with an unsalted 32-bit hash, and data lives only in one browser's localStorage.
- **CI and integration:** the executed pipeline only validates and archives three files; the Docker and Selenium stages and the second-phase work are not yet merged into main.
- **Testing and operations:** one browser, one machine and a single run; deployment is local only and monitoring is basic.

8.3 Future Enhancements

- **Backend and database:** server-side authentication with a database such as PostgreSQL or MongoDB and salted password hashing (bcrypt or Argon2) over HTTPS.
- **Advanced CI/CD and cloud:** activate the extended Jenkinsfile, publish JUnit reports, trigger builds with webhooks, push versioned images and deploy them to a cloud platform or Kubernetes.
- **Broader testing and monitoring:** unit, accessibility and cross-browser tests, a web health check, centralised logs, Prometheus and Grafana metrics, and build notifications.

REFERENCES

[1] Git Project, "Git Documentation." Available: https://git-scm.com/doc

[2] S. Chacon and B. Straub, Pro Git, 2nd ed., Apress. Available: https://git-scm.com/book/en/v2

[3] GitHub, "About pull requests," GitHub Docs. Available: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

[4] GitHub, "What is GitHub Pages?," GitHub Docs. Available: https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

[5] Jenkins Project, "Pipeline," Jenkins User Documentation. Available: https://www.jenkins.io/doc/book/pipeline/

[6] Jenkins Project, "Pipeline Syntax (Declarative Pipeline)." Available: https://www.jenkins.io/doc/book/pipeline/syntax/

[7] Docker Inc., "Dockerfile reference," Docker Docs. Available: https://docs.docker.com/reference/dockerfile/

[8] Docker Inc., "Docker Compose," Docker Docs. Available: https://docs.docker.com/compose/

[9] Docker Inc., "Using profiles with Compose," Docker Docs. Available: https://docs.docker.com/compose/how-tos/profiles/

[10] Docker Inc., "Networking in Compose," Docker Docs. Available: https://docs.docker.com/compose/how-tos/networking/

[11] Selenium Project, "WebDriver," Selenium Documentation. Available: https://www.selenium.dev/documentation/webdriver/

[12] Selenium Project, "Waiting Strategies," Selenium Documentation. Available: https://www.selenium.dev/documentation/webdriver/waits/

[13] pytest Developers, "pytest Documentation." Available: https://docs.pytest.org/en/stable/

[14] pytest Developers, "About fixtures." Available: https://docs.pytest.org/en/stable/explanation/fixtures.html

[15] MDN Web Docs, "Window: localStorage property." Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[16] MDN Web Docs, "Secure contexts." Available: https://developer.mozilla.org/en-US/docs/Web/Security/Secure_Contexts